

Program 2 – keyValue cList.

Assignment Objective: Build skills on C class creation while implementing a **circular list of** keyValue pairs.

Requirements:

- As with P1 List, a cListTemplate.cpp file is provided; it must be copied to cList.cpp. The formatting and ordering therein shall be maintained.
- The class properties:
 - Semantically, to the user, this class will behave the same as the **list** class in the list assignment. It has all the same member functions as **list**.
 - It has two new private member functions, virtualToPhysical() and physicalToVirtual(); see below.
 - It has two C Preprocessor macros: cListInc() and cListDec(); see below.
 - However, this will be implemented in a manner such that insert() will be O(1) as well as add() is O(1).
 - To do this, two new integer member variables will be defined: first and last.
 - first will be the index of the first item on the list.
 - last will be the index of the last item on the list.
 - When something is inserted, first will be decremented, and the value will be put at that index. Care must be taken that if first becomes negative, it should be set to the value of listCapacity-1. I.e., first will wrap around to the end of the array.
 - Similarly, when a value is added to the list, last will be incremented by 1, and the value will go there. If last is incremented to capacity, it will be set to 0 and the value will be inserted there.
 - Note, insertAt() and deleteAt() become more complicated. And performance is still O(n).
- Into two files, cList.h and cList.cpp, create a class/ADT called cList with the following members
 - You can start by copying your list.h to cList.h, as much of everything stays the same; only the class name changes, and then the following additions to cList.h.
 - Macros to be added to cList.h:
 - `#define cListInc(x) (x = (x + 1) % listCapacity)`
 - `#define cListDec(x) (x = (x + listCapacity - 1) % listCapacity)`
 - Private members:
 - Add the following as compared to the list.h to the private members:
 - `unsigned int first; // to track the first entry of the list`
 - `unsigned int last; // to track the last entry of the list`
 - `unsigned int virtualToPhysical(int virtualIndex) const // returns the list's physical index for the given virtual index.`
 - `unsigned int physicalToVirtual(int physicalIndex) const // returns the list's virtual index for the given physical index.`
 - `void setLast() - Uses first and listCount to set the "last" member variable.`
 - Public members:
 - These are all the same as the list ADT, save that the list type is now cList instead of list.
 - `cat()`, `insertAt()`, `deleteAt()`, `printItBackwards()`, and `cList(cList *)` are for part B. All else are in Part A.
- Copy the list Makefile and change the CHOOSE_LIST setting to be CHOOSE_CLIST. Change all the "list" targets to "cList" targets. It should create two executables: cListA and cListB.
- Demonstrate the correctness of the implementation by doing the following, using the same listMain.cpp that was in Program 1:
 - For Part A, just do: make cListA
 - For Part B, just do: make
 - Execute the code and inspect the results; if they are what you expect from your understanding of listMain.cpp, proceed to the submission; otherwise, inspect and fix your code as necessary and repeat these steps.

- The correct output is shown in the D2L files cListAcorrectOutput.txt and cListBcorrectOutput.txt
- **Submission process:**
 - For Part A, create the file listAoutput.txt using the following command in your environment:
 - ./cListA > cListAoutput.txt
 - For Part B, create the file listBoutput.txt using the following command in your environment:
 - ./cListB > cListBoutput.txt
 - **Printing:**
 - Your cList.h and cList.cpp shall be printed with line numbers, and with a constant pitch font; no variable pitch fonts will be accepted. Ensure that none of your code and/or comment lines wrap to a second line.
 - Submit your printouts in the following order: cList.h, cList.cpp.
 - NO screenshots will be accepted.
 - Bring the printouts to class when due.
 - **D2L submission:**
 - Submit a single zip file to D2L with the Makefile, cList.h, cList.cpp, and cListAoutput.txt or cListBoutput.txt, as appropriate.